

UNIVERSIDAD CATÓLICA BOLIVIANA

Diseño Web 2

Entrega 7B

Dashboards y Cierre MVP

FASE 3 — Inscripciones (Cierre)

Abril 2026

Proyecto: gestion-academica — ENTREGA FINAL

Objetivos de Aprendizaje

Al completar esta entrega final, el estudiante será capaz de:

- Crear dashboards personalizados por rol con estadísticas en tiempo real
- Implementar API Routes que calculan métricas agregadas (conteos, sumas, promedios)
- Reutilizar el componente StatsCard con datos dinámicos de múltiples fuentes
- Construir un grid de estadísticas responsivo con Tailwind CSS
- Comprender el patrón de página dashboard: datos calculados en el servidor, renderizados en el cliente
- Verificar la integridad del MVP completo con un checklist exhaustivo
- Documentar la arquitectura final del proyecto

Prerrequisitos

Antes de comenzar esta entrega final, asegúrate de haber completado:

- ✓ **TODAS las entregas anteriores: 1A a 7A funcionando correctamente**
- ✓ **Datos de prueba variados: al menos 3 estudiantes, 2 docentes, 5 materias**
- ✓ **Inscripciones mixtas: algunas activas, algunas retiradas**
- ✓ **Usuarios activos e inactivos en el sistema**

⚠ Entrega final: Esta es la última entrega del proyecto gestion-academica. Al terminar, tendrás un MVP completo con 3 roles, CRUD de usuarios y materias, sistema de inscripciones, vistas cruzadas y dashboards.

Teoría: Dashboards y Métricas

Anatomía de un Dashboard

Un dashboard es una página que resume el estado del sistema o la actividad del usuario. Tiene tres componentes principales:

Componente	Qué muestra	Ejemplo en nuestro proyecto
KPIs (Key Performance Indicators)	Números clave en cards destacadas	Total usuarios, materias, inscripciones
Resumen contextual	Información relevante para el rol	Mis materias inscritas, mis créditos
Acciones rápidas	Atajos a las funciones más usadas	Botón "Ir al catálogo", "Ver usuarios"

Dashboard por Rol

Cada rol tiene necesidades diferentes. Diseñaremos 3 dashboards distintos:

Diagrama: Dashboards por rol		
ESTUDIANTE	DOCENTE	ADMIN
Materias inscritas	Materias asignadas	Usuarios totales
Créditos activos	Estudiantes inscritos	Materias totales
Materias retiradas	Promedio por materia	Inscripc. totales
Acciones rápidas	Acciones rápidas	Acciones rápidas

Endpoint de Estadísticas

En lugar de hacer múltiples fetch desde el cliente, crearemos un endpoint /api/{rol}/stats que calcula todas las métricas en el servidor y las devuelve en un solo JSON. Esto tiene dos ventajas:

- Rendimiento: una sola llamada HTTP en vez de 3-4 llamadas paralelas
- Consistencia: todos los números se calculan en el mismo instante, sin race conditions

☑ **Patrón recomendado:** Para dashboards, es mejor calcular las métricas en el servidor y enviar un objeto plano con los números ya calculados. El cliente solo se encarga de mostrarlos.

Paso 1 — API Route: Estadísticas del Estudiante

1.1 — Crear la estructura

Terminal

```
mkdir -p app/api/estudiante/stats
```

1.2 — Crear la API Route

Crea el archivo `app/api/estudiante/stats/route.ts`:

`app/api/estudiante/stats/route.ts`

```
import { createClient } from "@lib/supabase/server";
import { NextResponse } from "next/server";

export async function GET() {
  try {
    const supabase = await createClient();
    const {
      data: { user },
      error: authError,
    } = await supabase.auth.getUser();

    if (authError || !user) {
      return NextResponse.json({ error: "No autenticado" }, { status: 401 });
    }

    const { data: perfil } = await supabase
      .from("usuarios_perfil")
      .select("rol")
      .eq("id", user.id)
      .single();

    if (perfil?.rol !== "estudiante") {
      return NextResponse.json({ error: "No autorizado" }, { status: 403 });
    }

    // Obtener inscripciones con créditos
    const { data: inscripciones } = await supabase
      .from("inscripciones")
      .select("estado, materia:materias!materia_id(creditos)")
      .eq("estudiante_id", user.id);

    const inscritas = inscripciones?.filter(
      (i) => i.estado === "inscrito"
    ) || [];
    const retiradas = inscripciones?.filter(
      (i) => i.estado === "retirado"
    ) || [];
  }
}
```

```

const creditosActivos = inscritis.reduce(
  (sum, i) => sum + ((i.materia as any)?.creditos || 0),
  0
);

// Total de materias disponibles
const { count: totalMaterias } = await supabase
  .from("materias")
  .select("id", { count: "exact", head: true });

return NextResponse.json({
  stats: {
    materiasInscritas: inscritis.length,
    materiasRetiradas: retiradas.length,
    creditosActivos,
    totalMateriasDisponibles: totalMaterias || 0,
  },
});
} catch (err) {
  console.error("Error inesperado:", err);
  return NextResponse.json({ error: "Error interno" }, { status: 500 });
}
}

```

☑ **count con head: true:** La opción { count: "exact", head: true } le dice a Supabase que solo cuente los registros sin devolver datos. Es mucho más eficiente que traer todos los registros y contar en JS.

Paso 2 — API Route: Estadísticas del Docente

2.1 — Crear la API Route

Terminal

```
mkdir -p app/api/docente/stats
```

Crea el archivo `app/api/docente/stats/route.ts`:

app/api/docente/stats/route.ts

```
import { createClient } from "@lib/supabase/server";
import { NextResponse } from "next/server";

export async function GET() {
  try {
    const supabase = await createClient();
    const {
      data: { user },
      error: authError,
    } = await supabase.auth.getUser();

    if (authError || !user) {
      return NextResponse.json({ error: "No autenticado" }, { status: 401 });
    }

    const { data: perfil } = await supabase
      .from("usuarios_perfil")
      .select("rol")
      .eq("id", user.id)
      .single();

    if (perfil?.rol !== "docente") {
      return NextResponse.json({ error: "No autorizado" }, { status: 403 });
    }

    // Materias del docente
    const { data: materias } = await supabase
      .from("materias")
      .select("id, creditos")
      .eq("docente_id", user.id);

    const totalMaterias = materias?.length || 0;
    const totalCreditos = materias?.reduce(
      (sum, m) => sum + m.creditos, 0
    ) || 0;

    // Inscritos en sus materias
    const materiaIds = materias?.map((m) => m.id) || [];
    let totalInscritos = 0;

    if (materiaIds.length > 0) {
```

```

    const { count } = await supabase
      .from("inscripciones")
      .select("id", { count: "exact", head: true })
      .in("materia_id", materiaIds)
      .eq("estado", "inscrito");
    totalInscritos = count || 0;
  }

  const promedio = totalMaterias > 0
    ? Math.round((totalInscritos / totalMaterias) * 10) / 10
    : 0;

  return NextResponse.json({
    stats: {
      totalMaterias,
      totalCreditos,
      totalInscritos,
      promedioEstudiantesPorMateria: promedio,
    },
  });
} catch (err) {
  console.error("Error inesperado:", err);
  return NextResponse.json({ error: "Error interno" }, { status: 500 });
}
}

```


Paso 3 — API Route: Estadísticas del Admin

3.1 — Crear la API Route

Terminal

```
mkdir -p app/api/admin/stats
```

Crea el archivo `app/api/admin/stats/route.ts`:

`app/api/admin/stats/route.ts`

```
import { createClient } from "@lib/supabase/server";
import { createClient as createServiceClient } from "@supabase/supabase-js";
import { NextResponse } from "next/server";

export async function GET() {
  try {
    const supabase = await createClient();
    const {
      data: { user },
      error: authError,
    } = await supabase.auth.getUser();

    if (authError || !user) {
      return NextResponse.json({ error: "No autenticado" }, { status: 401 });
    }

    const { data: perfil } = await supabase
      .from("usuarios_perfil")
      .select("rol")
      .eq("id", user.id)
      .single();

    if (perfil?.rol !== "admin") {
      return NextResponse.json({ error: "No autorizado" }, { status: 403 });
    }

    const supabaseAdmin = createServiceClient(
      process.env.NEXT_PUBLIC_SUPABASE_URL!,
      process.env.SUPABASE_SERVICE_ROLE_KEY!
    );

    // Conteos en paralelo
    const [usuarios, materias, inscripciones, inscritas] = await Promise.all([
      supabaseAdmin
        .from("usuarios_perfil")
        .select("id, rol, activo", { count: "exact" }),
      supabaseAdmin
        .from("materias")
        .select("id", { count: "exact", head: true }),
      supabaseAdmin
        .from("inscripciones")
```

```

        .select("id", { count: "exact", head: true }),
        supabaseAdmin
        .from("inscripciones")
        .select("id", { count: "exact", head: true })
        .eq("estado", "inscrito"),
    ]));

    // Desglose de usuarios por rol
    const perfiles = usuarios.data || [];
    const totalEstudiantes = perfiles.filter((u) => u.rol === "estudiante").length;
    const totalDocentes = perfiles.filter((u) => u.rol === "docente").length;
    const totalAdmins = perfiles.filter((u) => u.rol === "admin").length;
    const totalActivos = perfiles.filter((u) => u.activo).length;
    const totalInactivos = perfiles.filter((u) => !u.activo).length;

    return NextResponse.json({
      stats: {
        totalUsuarios: usuarios.count || 0,
        totalEstudiantes,
        totalDocentes,
        totalAdmins,
        totalActivos,
        totalInactivos,
        totalMaterias: materias.count || 0,
        totalInscripciones: inscripciones.count || 0,
        inscripcionesActivas: inscritas.count || 0,
      },
    });
  } catch (err) {
    console.error("Error inesperado:", err);
    return NextResponse.json({ error: "Error interno" }, { status: 500 });
  }
}

```

☑ **Promise.all:** Ejecutamos 4 consultas en paralelo con Promise.all. Esto es mucho más rápido que ejecutarlas secuencialmente (4 queries en paralelo vs 4 en serie).

Paso 4 — Componentes de Dashboard por Rol

4.1 — Dashboard del Estudiante

Crea el archivo `components/dashboard/estudiante/dashboard-estudiante.tsx`:

```
components/dashboard/estudiante/dashboard-estudiante.tsx
"use client";

import { useEffect, useState } from "react";
import { toast } from "sonner";
import StatsCard from "@/components/dashboard/stats-card";
import { Button } from "@/components/ui/button";
import { BookOpen, GraduationCap, XCircle, Library } from "lucide-react";
import { useRouter } from "next/navigation";

interface Stats {
  materiasInscritas: number;
  materiasRetiradas: number;
  creditosActivos: number;
  totalMateriasDisponibles: number;
}

export default function DashboardEstudiante() {
  const [stats, setStats] = useState<Stats | null>(null);
  const [loading, setLoading] = useState(true);
  const router = useRouter();

  useEffect(() => {
    const cargar = async () => {
      try {
        const res = await fetch("/api/estudiante/stats");
        const data = await res.json();
        if (res.ok) setStats(data.stats);
      } catch {
        toast.error("Error al cargar estadísticas");
      } finally {
        setLoading(false);
      }
    };
    cargar();
  }, []);

  if (loading) {
    return (
      <div className="flex items-center justify-center py-12">
        <p className="text-muted-foreground">Cargando dashboard...</p>
      </div>
    );
  }

  return (
    <div className="space-y-6">
      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-4">

```

```

    <StatsCard
      title="Materias Inscritas"
      value={stats?.materiasInscritas ?? 0}
      icon={BookOpen}
    />
    <StatsCard
      title="Créditos Activos"
      value={stats?.creditosActivos ?? 0}
      icon={GraduationCap}
    />
    <StatsCard
      title="Materias Retiradas"
      value={stats?.materiasRetiradas ?? 0}
      icon={XCircle}
    />
    <StatsCard
      title="Materias Disponibles"
      value={stats?.totalMateriasDisponibles ?? 0}
      icon={Library}
    />
  </div>

  { /* Acciones rápidas */ }
  <div className="flex gap-3">
    <Button onClick={() => router.push("/estudiante/materias")}>
      Ver Catálogo de Materias
    </Button>
    <Button variant="outline" onClick={() => router.push("/estudiante/inscripciones")}>
      Mis Inscripciones
    </Button>
  </div>
</div>
);
}

```

4.2 — Dashboard del Docente

Crea el archivo `components/dashboard/docente/dashboard-docente.tsx`:

```
components/dashboard/docente/dashboard-docente.tsx
"use client";

import { useEffect, useState } from "react";
import { toast } from "sonner";
import StatsCard from "@/components/dashboard/stats-card";
import { Button } from "@/components/ui/button";
import { BookOpen, Users, GraduationCap, BarChart3 } from "lucide-react";
import { useRouter } from "next/navigation";

interface Stats {
  totalMaterias: number;
  totalCreditos: number;
  totalInscritos: number;
  promedioEstudiantesPorMateria: number;
}

export default function DashboardDocente() {
  const [stats, setStats] = useState<Stats | null>(null);
  const [loading, setLoading] = useState(true);
  const router = useRouter();

  useEffect(() => {
    const cargar = async () => {
      try {
        const res = await fetch("/api/docente/stats");
        const data = await res.json();
        if (res.ok) setStats(data.stats);
      } catch {
        toast.error("Error al cargar estadísticas");
      } finally {
        setLoading(false);
      }
    };
    cargar();
  }, []);

  if (loading) {
    return (
      <div className="flex items-center justify-center py-12">
        <p className="text-muted-foreground">Cargando dashboard...</p>
      </div>
    );
  }

  return (
    <div className="space-y-6">
      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-4">
        <StatsCard
          title="Mis Materias"
          value={stats?.totalMaterias ?? 0}
          icon={BookOpen}
        />
        <StatsCard
          title="Total Créditos"
```

```

        value={stats?.totalCreditos ?? 0}
        icon={GraduationCap}
      />
      <StatsCard
        title="Estudiantes Inscritos"
        value={stats?.totalInscritos ?? 0}
        icon={Users}
      />
      <StatsCard
        title="Promedio por Materia"
        value={stats?.promedioEstudiantesPorMateria ?? 0}
        icon={BarChart3}
      />
    </div>

    <div className="flex gap-3">
      <Button onClick={() => router.push("/docente/materias")}>
        Ver Mis Materias
      </Button>
    </div>
  </div>
);
}

```

4.3 — Dashboard del Admin

Crea el archivo `components/dashboard/admin/dashboard-admin.tsx`:

```
components/dashboard/admin/dashboard-admin.tsx
"use client";

import { useEffect, useState } from "react";
import { toast } from "sonner";
import StatsCard from "@/components/dashboard/stats-card";
import { Button } from "@/components/ui/button";
import {
  Users, BookOpen, ClipboardList, UserCheck,
  UserX, GraduationCap, Shield, CheckCircle2,
} from "lucide-react";
import { useRouter } from "next/navigation";

interface Stats {
  totalUsuarios: number;
  totalEstudiantes: number;
  totalDocentes: number;
  totalAdmins: number;
  totalActivos: number;
  totalInactivos: number;
  totalMaterias: number;
  totalInscripciones: number;
  inscripcionesActivas: number;
}

export default function DashboardAdmin() {
  const [stats, setStats] = useState<Stats | null>(null);
  const [loading, setLoading] = useState(true);
  const router = useRouter();

  useEffect(() => {
    const cargar = async () => {
      try {
        const res = await fetch("/api/admin/stats");
        const data = await res.json();
        if (res.ok) setStats(data.stats);
      } catch {
        toast.error("Error al cargar estadísticas");
      } finally {
        setLoading(false);
      }
    };
    cargar();
  }, []);

  if (loading) {
    return (
      <div className="flex items-center justify-center py-12">
        <p className="text-muted-foreground">Cargando dashboard...</p>
      </div>
    );
  }

  return (
    <div className="space-y-6">
```

```

    { /* Fila 1: Usuarios */ }
    <div className="grid grid-cols-1 md:grid-cols-3 lg:grid-cols-6 gap-4">
      <StatsCard title="Usuarios" value={stats?.totalUsuarios ?? 0} icon={Users} />
      <StatsCard title="Estudiantes" value={stats?.totalEstudiantes ?? 0} icon={GraduationCap}
/>
      <StatsCard title="Docentes" value={stats?.totalDocentes ?? 0} icon={BookOpen} />
      <StatsCard title="Admins" value={stats?.totalAdmins ?? 0} icon={Shield} />
      <StatsCard title="Activos" value={stats?.totalActivos ?? 0} icon={UserCheck} />
      <StatsCard title="Inactivos" value={stats?.totalInactivos ?? 0} icon={UserX} />
    </div>

    { /* Fila 2: Materias e inscripciones */ }
    <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
      <StatsCard title="Materias" value={stats?.totalMaterias ?? 0} icon={BookOpen} />
      <StatsCard title="Inscripciones" value={stats?.totalInscripciones ?? 0}
icon={ClipboardList} />
      <StatsCard title="Inscripciones Activas" value={stats?.inscripcionesActivas ?? 0}
icon={CheckCircle2} />
    </div>

    { /* Acciones rápidas */ }
    <div className="flex gap-3 flex-wrap">
      <Button onClick={() => router.push("/admin/usuarios")}>Gestionar Usuarios</Button>
      <Button variant="outline" onClick={() => router.push("/admin/materias")}>Gestionar
Materias</Button>
      <Button variant="outline" onClick={() => router.push("/admin/inscripciones")}>Ver
Inscripciones</Button>
    </div>
  </div>
);
}

```


Paso 5 — Actualizar las Páginas de Dashboard

Reemplazaremos las 3 páginas placeholder de dashboard con las páginas reales.

5.1 — Dashboard Estudiante

Reemplaza `app/(dashboard)/estudiante/page.tsx`:

```
app/(dashboard)/estudiante/page.tsx
import DashboardEstudiante from "@/components/dashboard/estudiante/dashboard-estudiante";

export default function EstudianteDashboardPage() {
  return (
    <div className="flex flex-1 flex-col gap-4 p-4">
      <div>
        <h1 className="text-2xl font-bold tracking-tight">Dashboard</h1>
        <p className="text-muted-foreground">
          Bienvenido a tu panel de estudiante
        </p>
      </div>
      <DashboardEstudiante />
    </div>
  );
}
```

5.2 — Dashboard Docente

Reemplaza `app/(dashboard)/docente/page.tsx`:

```
app/(dashboard)/docente/page.tsx
import DashboardDocente from "@/components/dashboard/docente/dashboard-docente";

export default function DocenteDashboardPage() {
  return (
    <div className="flex flex-1 flex-col gap-4 p-4">
      <div>
        <h1 className="text-2xl font-bold tracking-tight">Dashboard</h1>
        <p className="text-muted-foreground">
          Bienvenido a tu panel de docente
        </p>
      </div>
      <DashboardDocente />
    </div>
  );
}
```

5.3 — Dashboard Admin

Reemplaza `app/(dashboard)/admin/page.tsx`:

app/(dashboard)/admin/page.tsx

```
import DashboardAdmin from "@components/dashboard/admin/dashboard-admin";

export default function AdminDashboardPage() {
  return (
    <div className="flex flex-1 flex-col gap-4 p-4">
      <div>
        <h1 className="text-2xl font-bold tracking-tight">Dashboard</h1>
        <p className="text-muted-foreground">
          Panel de administración del sistema
        </p>
      </div>
      <DashboardAdmin />
    </div>
  );
}
```

Resumen de Archivos

En esta entrega final creaste o modificaste los siguientes archivos:

Archivo	Acción	Descripción
app/api/estudiante/stats/route.ts	Nuevo	GET: métricas del estudiante
app/api/docente/stats/route.ts	Nuevo	GET: métricas del docente
app/api/admin/stats/route.ts	Nuevo	GET: métricas globales del sistema
components/dashboard/estudiante/dashboard-estudiante.tsx	Nuevo	4 StatsCard + acciones rápidas
components/dashboard/docente/dashboard-docente.tsx	Nuevo	4 StatsCard + acciones rápidas
components/dashboard/admin/dashboard-admin.tsx	Nuevo	9 StatsCard (2 filas) + acciones rápidas
app/(dashboard)/estudiante/page.tsx	Modificado	Reemplaza placeholder con DashboardEstudiante
app/(dashboard)/docente/page.tsx	Modificado	Reemplaza placeholder con DashboardDocente
app/(dashboard)/admin/page.tsx	Modificado	Reemplaza placeholder con DashboardAdmin

Verificación Final de Dashboards

Ejecuta `npm run dev` y prueba con los 3 roles:

A. Dashboard Estudiante

- ✓ Iniciar sesión como estudiante → redirige a `/estudiante`
- ✓ 4 StatsCard: Inscritas, Créditos, Retiradas, Disponibles
- ✓ Los números coinciden con los datos reales
- ✓ Botón "Ver Catálogo" navega a `/estudiante/materias`
- ✓ Botón "Mis Inscripciones" navega a `/estudiante/inscripciones`

B. Dashboard Docente

- ✓ Iniciar sesión como docente → redirige a `/docente`
- ✓ 4 StatsCard: Materias, Créditos, Inscritos, Promedio
- ✓ Los números coinciden con los datos reales
- ✓ Botón "Ver Mis Materias" navega a `/docente/materias`

C. Dashboard Admin

- ✓ Iniciar sesión como admin → redirige a `/admin`
- ✓ Fila 1: 6 StatsCard de usuarios (total, por rol, activos/inactivos)
- ✓ Fila 2: 3 StatsCard (materias, inscripciones, activas)
- ✓ 3 botones de acción rápida funcionan correctamente

D. Grid responsivo

- ✓ En móvil: 1 card por fila
- ✓ En tablet: 2-3 cards por fila
- ✓ En desktop: layout completo según el grid definido

Checklist Final — Cierre del MVP

Verifica que TODAS las funcionalidades del MVP estén operativas:

Fase 1 — Fundación

- ✓ 1A: Proyecto Next.js 16 creado y corriendo con Turbopack
- ✓ 1B: shadcn/ui instalado, BD creada con 5 tablas en Supabase
- ✓ 2A: Clientes Supabase (browser + server), proxy.ts protegiendo rutas
- ✓ 2B: Login y registro funcionales, redirect por rol
- ✓ 3A: Sidebar dinámico por rol, layouts anidados
- ✓ 3B: Header con avatar, logout, hook use-usuario

Fase 2 — Gestión de Usuarios y Materias

- ✓ 4A: Página de perfil con Zod + React Hook Form
- ✓ 4B: Panel admin de usuarios (tabla, filtros, cambio rol, activar/desactivar)
- ✓ 5A: API y listado de materias, formulario de creación con Dialog
- ✓ 5B: Edición y eliminación de materias, Dialog reutilizable
- ✓ 5C: Vista docente (Mis Materias) y estudiante (Catálogo) con MateriaCard

Fase 3 — Inscripciones

- ✓ 6A: API de inscripción, botón activo en catálogo, RLS configurado
- ✓ 6B: Página Mis Inscripciones, retiro con confirmación, badges de estado
- ✓ 7A: Vista docente con estudiantes por materia, vista admin de todas las inscripciones
- ✓ 7B: Dashboards con stats reales para los 3 roles

Funcionalidades Transversales

- ✓ Autenticación SSR con cookies (getAll/setAll)
- ✓ Protección de rutas con proxy.ts y redirección por rol
- ✓ API Routes protegidas con verificación de rol
- ✓ RLS configurado en inscripciones, materias y usuarios_perfil
- ✓ Toast de Sonner para feedback de todas las operaciones
- ✓ Validación con Zod en servidor y cliente
- ✓ Componentes reutilizables: StatsCard, MateriaCard
- ✓ Patrón consistente: Server Component + Client Component
- ✓ Responsive design en todas las páginas

Arquitectura Final del Proyecto

Estructura de Archivos Completa

gestion-academica/ – Estructura final

```

app/
├── layout.tsx, page.tsx, globals.css
├── (auth)/
│   ├── layout.tsx, login/page.tsx, registro/page.tsx
│   └── auth/callback/route.ts
├── (dashboard)/
│   ├── layout.tsx
│   ├── estudiante/ (page, materias, inscripciones, perfil)
│   ├── docente/ (page, materias, perfil)
│   └── admin/ (page, usuarios, materias, inscripciones)
└── api/
    ├── auth/ (registro, perfil)
    ├── estudiante/ (materias, inscripciones, inscripciones/[id], stats)
    ├── docente/ (materias, inscripciones, stats)
    └── admin/ (usuarios, usuarios/[id], materias, materias/[id],
               docentes, inscripciones, stats)
  
```

Componentes

```

components/
├── ui/ (shadcn: button, input, label, card, form, table, badge,
│       select, dialog, alert-dialog, sidebar, dropdown-menu,
│       avatar, tooltip, separator, sonner, textarea)
├── auth/ (login-form, registro-form)
├── dashboard/
│   ├── app-sidebar.tsx, header.tsx, stats-card.tsx,
│   │   perfil-form.tsx, materia-card.tsx
│   ├── admin/ (usuarios-table, materias-table,
│   │           inscripciones-table, dashboard-admin)
│   ├── docente/ (mis-materias, dashboard-docente)
│   └── estudiante/ (catalogo-materias, mis-inscripciones,
│                  dashboard-estudiante)
  
```

Resumen de Entregas

Fase	Entregas	Total archivos	Concepto principal
1 - Fundación	1A, 1B, 2A, 2B, 3A, 3B	~20	Setup, Auth SSR, Layouts, Sidebar
2 - Gestión	4A, 4B, 5A, 5B, 5C	~15	CRUD, Zod, Tables, Dialogs, Cards
3 - Inscripciones	6A, 6B, 7A, 7B	~15	Relaciones M:N, RLS, Vistas cruzadas, Dashboards

Métrica del proyecto	Valor
Total de entregas	15 (7 principales con sub-entregas A/B/C)
API Routes creadas	14 endpoints (GET, POST, PUT, DELETE)
Componentes creados	~15 componentes propios + ~15 shadcn/ui
Tablas en BD	5 (usuarios_perfil, materias, inscripciones, calificaciones*, documentos*)
Políticas RLS	~7 políticas para 3 tablas
Stack tecnológico	Next.js 16, TypeScript, Supabase, shadcn/ui, Zod, RHF

i Post-MVP: Las tablas calificaciones y documentos_academicos se crearon en la BD (Entrega 1B) pero no se implementaron en la interfaz. Quedan como extensión natural para un proyecto futuro.

Ejercicio Final

Para cerrar el proyecto, implementa al menos 2 de las siguientes mejoras:

1. Gráfico de inscripciones: Instala recharts (npm install recharts) y agrega un gráfico de barras al dashboard del admin que muestre inscripciones por materia. Usa los datos de la API de stats o crea un nuevo endpoint que devuelva los conteos agrupados.
2. Modo oscuro: shadcn/ui soporta modo oscuro. Agrega un botón toggle en el header que cambie entre tema claro y oscuro. Usa next-themes (npm install next-themes) y el ThemeProvider en app/layout.tsx.
3. Página 404 personalizada: Crea app/not-found.tsx con un diseño amigable que use componentes de shadcn/ui. Incluye un botón "Volver al inicio" que redirija al dashboard según el rol del usuario.
4. Deploy en Vercel: Despliega la aplicación en Vercel (vercel.com). Configura las 3 variables de entorno en el panel de Vercel. Comparte la URL desplegada como parte de tu entrega final.

☒ **Entrega final:** Presenta tu proyecto funcionando en clase: muestra login con los 3 roles, navega por todas las funcionalidades, y explica las decisiones técnicas que tomaste. Prepara capturas de los 3 dashboards y de las funcionalidades principales.

¡Felicidades! — MVP Completado

Has completado exitosamente las 15 entregas del proyecto gestion-academica. En este recorrido aprendiste:

- Arquitectura moderna: Next.js 16 con App Router, Server y Client Components
- Autenticación SSR: cookies, proxy.ts, redirección por rol
- Base de datos: PostgreSQL con Supabase, JOINS, RLS, constraints
- Validación: Zod + React Hook Form en cliente y servidor
- UI profesional: shadcn/ui con 15+ componentes, Tailwind CSS responsive
- API REST: 14 endpoints con protección de rol y manejo de errores
- CRUD completo: usuarios y materias con crear, leer, editar, eliminar
- Relaciones M:N: inscripciones como tabla intermedia, estados, soft delete
- Vistas cruzadas: datos compartidos entre roles con RLS
- Dashboards: métricas en tiempo real calculadas en el servidor

Estos conocimientos son la base para construir cualquier aplicación web moderna. El siguiente paso es seguir practicando: agrega las calificaciones, implementa notificaciones, o despliega tu proyecto en la nube.

FIN DEL PROYECTO — gestion-academica MVP

Universidad Católica Boliviana • Diseño Web 2 • Abril 2026

15 entregas • 3 fases • ~50 archivos • 1 MVP completo